OCR H446 / PAPER 2 / UNIT 2.1

Computational thinking

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

01 Abstraction: keep what matters	6
02 Abstraction: models and assumptions	10
03 Thinking ahead: preconditions	14
04 Thinking ahead: caching	18
05 Thinking ahead: reuse	22
06 Thinking procedurally	26
07 Thinking logically	30
08 Thinking concurrently: dependencies	34
09 Thinking concurrently: trade-offs	38

MASTER CONTENTS

Choose a learning cycle

Select a row to jump to the four-page revision cycle. Each topic has an explanation, an original worked example, questions and a separate self-mark page.

2.1.1(a,b)	Thinking abstractly / Keep What Matters	pp. 6-9
2.1.1(c,d)	Thinking abstractly / Test The Assumptions	pp. 10-13
2.1.2(a,b)	Thinking ahead / Inputs And Preconditions	pp. 14-17
2.1.2(c)	Thinking ahead / Caching	pp. 18-21
2.1.2(d)	Thinking ahead / Reusable Components	pp. 22-25
2.1.3(a-d)	Thinking procedurally / Decompose And Order	pp. 26-29
2.1.4(a-c)	Thinking logically / Conditions And Paths	pp. 30-33
2.1.5(a)	Thinking concurrently / Dependencies	pp. 34-37
2.1.5(b)	Thinking concurrently / Trade-Offs	pp. 38-41

NAVIGATION TIP

Use the PDF bookmark panel or click a topic above. A live MCQ practice link is provided after each cycle.

INDEPENDENT REVISION

How to use each four-page cycle

01

UNDERSTAND

Read the explanation. Trace the diagram. Recall the highlighted vocabulary.

02

MODEL

Follow the complete worked example, not just the final output.

03

TRY

Work independently in black pen. Explain, trace or evaluate as asked.

04

CHECK

Self-mark in green pen. Correct one weakness and explain the fix.

STUDY TIP

Use the CHECK page after completing TRY. Correct errors and explain each change.

2.1 / ELEMENTS OF COMPUTATIONAL THINKING

Understand it. Model it. Try it. Check it.

The five official OCR thinking areas connect to practical algorithm design. Each subtopic here uses scenarios, diagrams, fully traced examples and independent practice.

2.1.1 Thinking abstractly

2.1.2 Thinking ahead

2.1.3 Thinking procedurally

2.1.4 Thinking logically

2.1.5 Thinking concurrently

WHAT REMAINS UNCHANGED

This is an independent chapter candidate.

2.1 / SPECIFICATION CHECKLIST

Coverage before page count

Nine four-page cycles cover all five numbered 2.1 topics and their 17 lettered subpoints. Tick only after you can apply the idea in an unfamiliar scenario.

SPEC	SPECIFICATION REQUIREMENT	PAGES
2.1.1(a,b)	Nature and purpose of abstraction	6-9
2.1.1(c,d)	Abstraction versus reality; devising models	10-13
2.1.2(a,b)	Inputs/outputs and preconditions	14-17
2.1.2(c)	Caching: nature, benefits and drawbacks	18-21
2.1.2(d)	Reusable program components	22-25
2.1.3(a-d)	Components, ordering, sub-procedures	26-29
2.1.4(a-c)	Decision points, logic and program paths	30-33
2.1.5(a)	Parts that can be tackled together	34-37
2.1.5(b)	Concurrent benefits and trade-offs	38-41

SPECIFICATION SOURCE

OCR H446 A Level Computer Science specification v2.7 (2024), section 2c, 2.1. Wording is summarised here; consult OCR for the official requirements.

2.1.1(a,b) / SEE IT / CONCEPT AND VISUAL

Keep the features the problem actually needs

Abstraction deliberately removes detail while retaining information that is relevant to a particular task.

MODEL

A purposeful simplified representation.

KEEP

Objects, links, costs and constraints needed to answer the question.

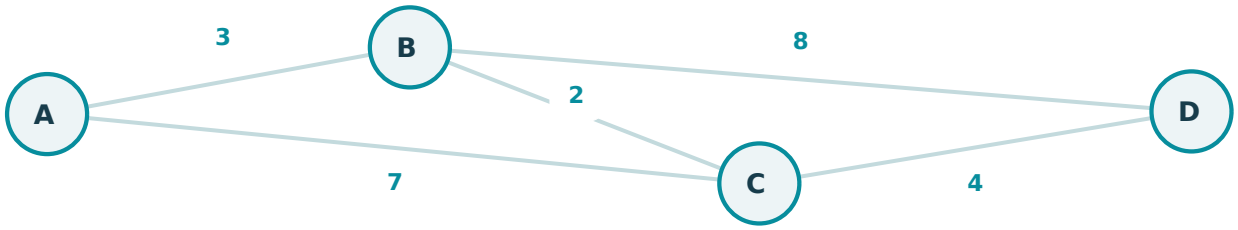
IGNORE

Appearance, decoration and background detail that do not affect the answer.

CHECK

Can the model still answer the original question?

A CITY MAP FOR FINDING THE SHORTEST ROUTE



FROM THE DIAGRAM TO EXAM LANGUAGE

A route planner can represent stations as vertices and direct journeys as weighted edges. Train colour and station-wall artwork usually do not change the shortest-route calculation.

WORDS THAT EARN MARKS

Abstraction	Relevant detail	**Model**	Representation
Vertex	Location	**Edge weight**	Journey cost

EXAM TRAP

Abstraction does not mean deleting as much as possible. Remove only what is irrelevant to the stated purpose.

2.1.1(a,b) / WORKED EXAMPLE / STEP BY STEP

From a real network to a weighted graph

A pupil needs the fastest route from A to D. Travel times: A-B=3, A-C=7, B-C=2, B-D=8, C-D=4 minutes.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Objective	**Minimise travel time**	The task defines which information is relevant.
2 / Nodes	**A, B, C, D**	A station is represented by a labelled vertex.
3 / Edges	**AB 3, AC 7, BC 2, BD 8, CD 4**	Only direct journeys become weighted edges.
4 / Compare	**A-B-C-D = 3+2+4 = 9**	This route uses an intermediate stop.
5 / Alternative	**A-C-D = 7+4 = 11**	A shorter-looking route can take longer.
6 / Decision	**Choose A-B-C-D: 9 minutes**	The model preserves the measure we optimise.

TRANSFER IT TO AN EXAM ANSWER

What we have not modelled: station colour, ticket design, seat layout or weather. Add them only if the question changes.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.1(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define abstraction and state two types of detail that a shortest-route model should retain.

Q2 / 5 MARKS

Explain how a weighted graph abstracts the rail network above. Give two deliberately omitted details and justify one omission.

Q3 / 6 MARKS

A passenger now needs a wheelchair-accessible journey. Evaluate the original model and explain how to modify it to meet this changed requirement.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.1(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Abstraction is simplifying a situation to its relevant features [1]. Keep the connected stations/journeys [1] and a route cost such as travel time [1].

Q2 / 5

Stations represented as vertices [1]; direct journeys as edges [1]; travel minutes as edge weights [1]; irrelevant details might include paint colour or advertisements [1]; these are omitted because they do not change journey time [1].

Q3 / 6

Existing weights and connections support time optimisation [1], but the model omits accessibility [1]. Add a step-free attribute to stations or journeys [1]. Exclude inaccessible edges or apply an accessibility constraint [1]. Recalculate among valid routes [1]. Explain that adding only necessary features retains the value of abstraction [1].

COMMON MISCONCEPTION

Abstraction does not mean deleting as much as possible. Remove only what is irrelevant to the stated purpose.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.1(c,d) / SEE IT / CONCEPT AND VISUAL

A model is useful because it is not reality

Every abstraction introduces assumptions. A good model is accurate enough for its purpose, not a copy of the real world.

REAL WORLD

Hundreds of events and changing circumstances.

ASSUMPTION

A declared simplification, such as constant journey time.

MODEL

A smaller set of facts with clear relationships.

VALIDATE

Test the model against realistic and boundary scenarios.

DESIGN A LIBRARY LOAN MODEL

Real loan: ID / date / cover / shelf



Model: loan / due / return status

Ignored here: cover colour, shelf decoration

FROM THE DIAGRAM TO EXAM LANGUAGE

For overdue reminders, the system needs loan ID, borrower contact, book identity and due date. Cover colour and shelf decoration are irrelevant; renewals and lost books are not.

WORDS THAT EARN MARKS

- **Assumption** Simplification
- **Scope** Included cases
- **Validation** Check suitability
- **Constraint** Requirement

EXAM TRAP

A model that works for an ordinary case is not automatically suitable for every real-world case.

2.1.1(c,d) / WORKED EXAMPLE / STEP BY STEP

Model late returns without inventing unnecessary data

A school library sends an alert when a book is overdue. Today is day 20; loan L7 has due day 17. Grace period is zero.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Required data	**Loan L7; due_day=17; today=20**	Retain the dates for the overdue rule.
2 / Condition	**today > due_day**	Define what counts as late.
3 / Calculation	**20 - 17 = 3 days**	Use integer day values in this simplified example.
4 / Output	**L7: overdue by 3 days**	Expose information needed by the user.
5 / Boundary	**today = due_day => not overdue**	Test the comparison at the boundary.
6 / Limitation	**Calendar/holidays ignored**	The simplified day numbers need real-date rules in production.

TRANSFER IT TO AN EXAM ANSWER

The solution should also test returned, renewed and missing loans. A model that ignores relevant exceptions may be misleading.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.1(c,d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Explain one difference between a real-world loan and its computer model. Give two fields needed for an overdue reminder.

Q2 / 5 MARKS

Design the data and the condition needed to send reminders for an overdue item. Include a boundary test.

Q3 / 6 MARKS

The school adds a rule: no reminders during closure days. Evaluate the current abstract model and explain how you would improve and test it.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.1(c,d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

A model includes selected aspects rather than every real-world detail [1]; due date [1] and borrower identifier/contact or return status [1].

Q2 / 5

Retain the loan and borrower identifiers [1], due date [1], today/current date [1]; send only if today > due date and not returned [1]; test today = due date, expecting no reminder [1].

Q3 / 6

The original model cannot distinguish closure days [1]. Include calendar/closure information [1]. Define how the policy adjusts due dates or reminder timing [1]. Apply the rule consistently [1]. Test a due date on a closure day and a normal day [1]. Validate actual notifications against school policy [1].

COMMON MISCONCEPTION

A model that works for an ordinary case is not automatically suitable for every real-world case.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(a,b) / SEE IT / CONCEPT AND VISUAL

Before you code, define what must be true

Thinking ahead identifies inputs, outputs, prior conditions and likely consequences before implementation.

INPUTS

Values supplied: user, event, quantity.

PRECONDITIONS

Conditions required for the operation to succeed.

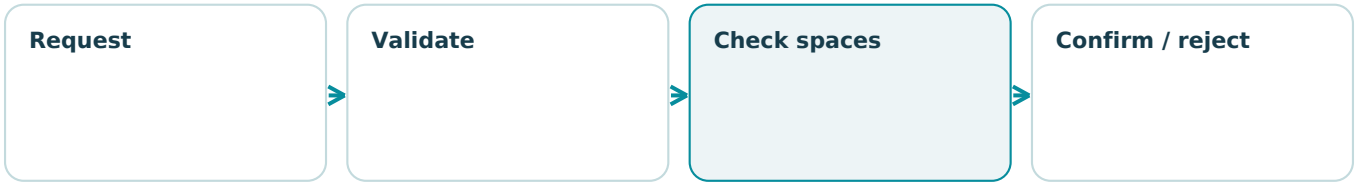
PROCESS

Validation, checking capacity and updating state.

OUTPUTS

A confirmed booking or an explained rejection.

A BOOKING FUNCTION: INPUT TO OUTPUT



FROM THE DIAGRAM TO EXAM LANGUAGE

A request for two seats is not sufficient on its own. It must have a valid event, a non-negative quantity and enough remaining capacity. Only a successful request updates seats.

WORDS THAT EARN MARKS

Input	Supplied value	**Output**	Produced result
Precondition	Must hold first	**Postcondition**	Must hold after

EXAM TRAP

A precondition describes what must hold before an operation; an output describes the result, not a prerequisite.

2.1.2(a,b) / WORKED EXAMPLE / STEP BY STEP

Specify a booking operation before writing code

A workshop has capacity 24, with 19 places already booked. One valid user requests 4 places.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Inputs	**capacity 24, booked 19, request 4**	Record the relevant state and request.
2 / Preconditions	**request > 0, user/event valid**	Do not assume valid data.
3 / Available	**24 - 19 = 5**	Compute the remaining capacity.
4 / Branch	**4 <= 5 : true**	The request fits.
5 / Update	**booked = 19 + 4 = 23**	Only update after passing all checks.
6 / Output	**Confirmation; remaining = 1**	Return a meaningful success result.

TRANSFER IT TO AN EXAM ANSWER

For a request of 6 places, reject and keep booked at 19; a failed operation must not modify the booking count.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(a,b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Identify two inputs to a seat-booking system and one appropriate output.

Q2 / 5 MARKS

State suitable preconditions for accepting a request. Explain the result for 19 booked places, capacity 24 and request 6.

Q3 / 6 MARKS

Design and explain a booking algorithm that never exceeds capacity, including what happens when an input fails validation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(a,b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Event ID [1]; requested number of seats [1]; output could be confirmation or a reason for rejection [1].

Q2 / 5

Event exists [1]; request is positive integer [1]; valid booking/user or suitable other precondition [1]; available is 24-19 = 5 [1]; request 6 is rejected with no state change [1].

Q3 / 6

Validate supplied fields [1]; calculate available capacity [1]; check request <= available [1]; if true update bookings and confirm [1]; otherwise return rejection with no update [1]; protect the check/update operation against competing requests [1].

COMMON MISCONCEPTION

A precondition describes what must hold before an operation; an output describes the result, not a prerequisite.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(c) / SEE IT / CONCEPT AND VISUAL

A cached answer can become the wrong answer

A cache stores a copy of data or a result for reuse. Reduced latency is valuable only if stale data is handled.

CACHE HIT

Reusable copy found: answer may be quicker.

CACHE MISS

Retrieve or compute the value, then store it.

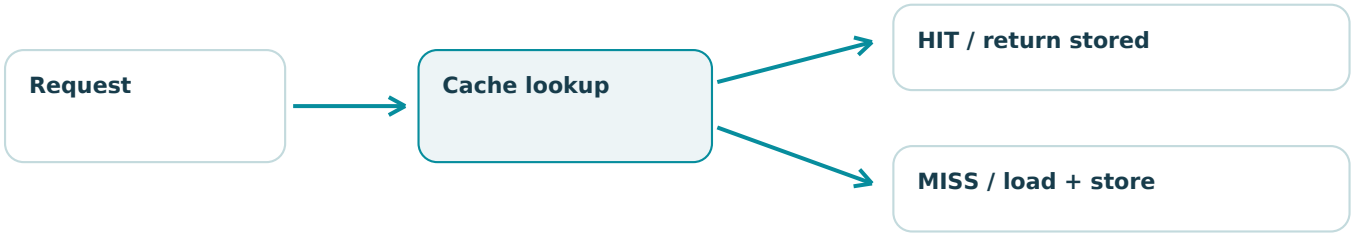
BENEFIT

Less repeated work and lower response time.

COST

Storage, invalidation, complexity and stale values.

A SIMPLE READ-THROUGH CACHE



FROM THE DIAGRAM TO EXAM LANGUAGE

Caching results of an expensive timetable search can save computation. If room allocations change, a cache containing the old timetable must expire or be invalidated.

WORDS THAT EARN MARKS

- **Cache** Stored copy
- **Hit** Value present
- **Miss** Value absent
- **Invalidation** Discard stale copy

EXAM TRAP

A cache is not always faster overall, and a cache hit does not guarantee that the answer is current.

2.1.2(c) / WORKED EXAMPLE / STEP BY STEP

Cache an expensive timetable lookup

A room timetable is requested three times. Uncached lookup takes 120 ms; a cache hit takes 8 ms. Ignore cache-management overhead for this exercise.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Request 1	**Miss => database 120 ms**	Nothing is cached yet.
2 / Save	**Cache key room:R4**	Associate a key with the returned timetable.
3 / Request 2	**Hit => 8 ms**	Return a stored copy.
4 / Request 3	**Hit => 8 ms**	Reuse the stored result again.
5 / Total	**120 + 8 + 8 = 136 ms**	Three uncached reads would total 360 ms.
6 / Update	**Room change => invalidate R4**	Old answers must no longer be reused.

TRANSFER IT TO AN EXAM ANSWER

Saving = 224 ms for these three reads under the stated timings. The real gain depends on cache hit rate and update frequency.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define caching, cache hit and cache miss.

Q2 / 5 MARKS

Calculate the time saved by the three requests above and explain why the result is conditional on the assumptions.

Q3 / 6 MARKS

Evaluate using a cache for live exam-room changes. Discuss responsiveness, invalidation and consequences of stale data.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Caching stores a reusable copy [1]; hit means the requested value is present [1]; miss means it is not present and must be obtained/computed [1].

Q2 / 5

Uncached 3 x 120 = 360 ms [1]; cached 120 + 8 + 8 = 136 ms [1]; saving 224 ms [1]; assumes the subsequent requests hit [1]; excludes changes, overhead and other latency [1].

Q3 / 6

A hit provides faster access [1] and reduces repeat queries [1], but copies can become stale [1]. Changes require invalidation or expiration [1]. Incorrect room assignments can misdirect students [1]. Use a suitable freshness policy or fetch authoritative data when essential [1].

COMMON MISCONCEPTION

A cache is not always faster overall, and a cache hit does not guarantee that the answer is current.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.2(d) / SEE IT / CONCEPT AND VISUAL

Reuse a component, not a copied mistake

A reusable procedure or function has a defined purpose, inputs, outputs and behaviour that can be tested independently.

ONE PURPOSE

Keep the function focused and cohesive.

INTERFACE

Clear parameters, returned value and assumptions.

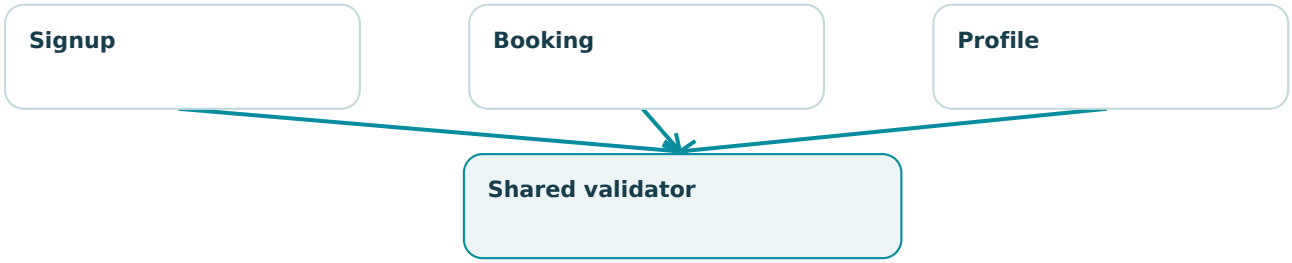
REUSE

Call the same validated component in several features.

TEST

Boundary and invalid-input tests protect every caller.

ONE VALIDATOR USED BY THREE SCREENS



FROM THE DIAGRAM TO EXAM LANGUAGE

Sign-up, booking and profile screens can all call a shared email validator. A code change then affects every caller, so its contract and tests matter.

WORDS THAT EARN MARKS

Function	Reusable unit	**Parameter**	Passed input
Return value	Produced result	**Contract**	Expected behaviour

EXAM TRAP

Reuse is not copying a block into several places; copied blocks drift apart and require multiple fixes.

2.1.2(d) / WORKED EXAMPLE / STEP BY STEP

Create an independently testable quantity validator

A function validQuantity(q, maxSeats) returns true only when q is a positive integer no greater than maxSeats.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Contract	validQuantity(q, maxSeats) -> Boolean	Specify parameters and output.
2 / Basic	q is an integer	Reject text, decimals and booleans as quantities.
3 / Lower bound	q >= 1	Reject zero and negative numbers.
4 / Upper bound	q <= maxSeats	Reject requests above the permitted maximum.
5 / Test	q=4,max=5 => true	Normal valid case.
6 / Boundaries	q=0 => false; q=5 => true	Confirm lower and upper edges.

TRANSFER IT TO AN EXAM ANSWER

The validator does not itself reserve seats or check live availability. Other components can call it before their own state updates.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.2(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State three benefits or design features of a reusable program component.

Q2 / 5 MARKS

Write pseudocode for validQuantity(q, maximum) using an integer check and inclusive range 1 to maximum.

Q3 / 6 MARKS

Explain why shared validation requires a clear interface and give a suitable set of normal, boundary and invalid tests.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.2(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Any three: one maintained implementation [1]; reduced duplication [1]; consistent behaviour [1]; defined parameters/interface [1]; independent testing [1].

Q2 / 5

Function accepts two parameters [1]; rejects non-integer q [1]; checks q >= 1 [1]; checks q <= maximum [1]; returns Boolean without modifying bookings [1].

Q3 / 6

The interface defines inputs and output [1]; callers rely on consistent rules [1]. Normal q=3,max=5 => true [1]; lower boundary q=1 => true [1]; invalid q=0 or 6 => false [1]; non-integer/text input => false [1].

COMMON MISCONCEPTION

Reuse is not copying a block into several places; copied blocks drift apart and require multiple fixes.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.3(a-d) / SEE IT / CONCEPT AND VISUAL

Break a problem into steps that can run

Procedural thinking identifies components, orders their operations and extracts useful sub-procedures.

DECOMPOSE

Identify manageable subproblems and data.

SEQUENCE

Decide which steps depend on earlier work.

SUB-PROCEDURE

Give repeated or cohesive operations a name.

INTERFACE

Pass required values between procedures.

A LIBRARY RETURN IS A SEQUENCE



FROM THE DIAGRAM TO EXAM LANGUAGE

The database lookup must happen before the update. Printing a receipt should use the confirmed updated state, not an assumed success.

WORDS THAT EARN MARKS

Decomposition	Smaller parts	**Sequence**	Ordered steps
Procedure	Named actions	**Dependency**	Needs earlier result

EXAM TRAP

Decomposition is not the same as simply listing code in order; the components must have meaningful responsibilities.

2.1.3(a-d) / WORKED EXAMPLE / STEP BY STEP

Design a book-return procedure

An assistant scans book B42; an active loan exists. The system must close the loan and provide a receipt.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Capture	**bookId <- scan()**	Obtain the identity of the book.
2 / Validate	**findActiveLoan(bookId)**	Get the current loan record.
3 / Branch	**if loan not found => explain error**	Stop safely if there is no active loan.
4 / Update	**closeLoan(loanId)**	Commit the return before confirming it.
5 / Output	**makeReceipt(loanId)**	Show user the confirmed result.
6 / Reuse	**lookup(), close(), receipt()**	Use named parts with clear responsibilities.

TRANSFER IT TO AN EXAM ANSWER

If update fails, do not issue a successful receipt. A success message belongs after the confirmed state change.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.3(a-d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define decomposition, sequencing and a sub-procedure in the context of a book-return system.

Q2 / 5 MARKS

List five suitably ordered steps to process a return, including one validation and one failure outcome.

Q3 / 6 MARKS

Outline a modular solution to this problem, explaining data passed between components and two important dependencies.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.3(a-d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Decomposition divides return processing into smaller tasks [1]; sequencing puts operations in a necessary order [1]; a sub-procedure is a named part such as findActiveLoan() [1].

Q2 / 5

Read/scanned book ID [1]; validate/find active loan [1]; reject if none [1]; update the record [1]; issue receipt only after confirmed success [1].

Q3 / 6

Identify scan or capture component [1]; loan lookup component [1]; update component [1]; receipt/output component [1]; pass book ID or loan ID between components [1]; lookup before update and successful update before receipt [1].

COMMON MISCONCEPTION

Decomposition is not the same as simply listing code in order; the components must have meaningful responsibilities.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.4(a-c) / SEE IT / CONCEPT AND VISUAL

Every decision creates a path through the program

Logical thinking identifies choice points, expresses conditions precisely and traces their consequences.

DECISION POINT

Where a choice changes what happens next.

CONDITION

An expression that evaluates to true or false.

BRANCH

A path taken when the condition matches.

TEST

Use boundary and contrasting values to test paths.

A BOOLEAN DECISION THAT CAN BE TRACED



FROM THE DIAGRAM TO EXAM LANGUAGE

For a restricted workshop, admission requires age >= 16 AND validID = true. Being 16 does not override a failed identity check; both conditions matter.

WORDS THAT EARN MARKS

- **Boolean** True or false
- **AND** Both true
- **OR** Either true
- **Branch** Selected path

EXAM TRAP

An inclusive condition >= 16 is different from > 16. Tests should include the exact boundary value.

2.1.4(a-c) / WORKED EXAMPLE / STEP BY STEP

Construct and trace a compound condition

Entry rule: age >= 16 AND validID. Trace all four cases below with an explicit result.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Condition	**eligible = (age >= 16) AND validID**	Both requirements must hold.
2 / Case A	**age 15, ID yes => false**	Age fails.
3 / Case B	**age 16, ID no => false**	ID fails at the age boundary.
4 / Case C	**age 16, ID yes => true**	Both conditions are satisfied.
5 / Case D	**age 19, ID yes => true**	A value above the boundary is valid.
6 / Action	**if eligible then admit else reject**	Only one path executes.

TRANSFER IT TO AN EXAM ANSWER

Parentheses make the intended logic clear. Change AND to OR and the rule would admit an under-age person with ID, which is incorrect.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.4(a-c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State the purpose of a condition, a Boolean operator and a branch.

Q2 / 5 MARKS

Complete the admission rule in pseudocode and trace age=16/validID=false and age=17/validID=true.

Q3 / 6 MARKS

Evaluate a flawed program that uses OR instead of AND. Show two counterexamples and explain a test strategy.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.4(a-c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Condition evaluates to true/false [1]; Boolean operator combines or negates conditions [1]; branch chooses subsequent execution path [1].

Q2 / 5

eligible <- (age >= 16) AND validID [1]; if eligible then admit [1] else reject [1]; (16,false) rejects [1]; (17,true) admits [1].

Q3 / 6

OR allows entry when only one condition holds [1]. Age 15 with validID=true is incorrectly admitted [1]. Age 18 with validID=false is also incorrectly admitted [1]. AND requires both [1]. Test at age 15/16 [1] with both ID states to cover all logical branches [1].

COMMON MISCONCEPTION

An inclusive condition >= 16 is different from > 16. Tests should include the exact boundary value.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.5(a) / SEE IT / CONCEPT AND VISUAL

Parallel work needs independent tasks

Thinking concurrently identifies tasks that may proceed together, while respecting dependencies between results.

INDEPENDENT

Can begin without needing the other result.

DEPENDENT

Must wait until prerequisite work is complete.

JOIN

Wait for all needed results before the next step.

PLAN

A dependency diagram reveals available parallelism.

A SIMPLE DEPENDENCY MAP



FROM THE DIAGRAM TO EXAM LANGUAGE

After a batch begins, file A and file B can be parsed concurrently. Comparing their results must wait for both parses to complete.

WORDS THAT EARN MARKS

- **Concurrency** — Overlapping progress
- **Dependency** — Must come first
- **Parallelism** — Simultaneous execution
- **Join** — Wait for results

EXAM TRAP

Concurrency is not a promise that every line runs simultaneously; dependent tasks still require ordering.

2.1.5(a) / WORKED EXAMPLE / STEP BY STEP

Calculate the earliest finish under two workers

Task Load=2 s. After Load, Parse A=5 s and Parse B=4 s may run together. Compare=2 s needs both parses.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Start	**Load: time 0-2 s**	All later tasks require loaded inputs.
2 / Branch A	**Parse A: time 2-7 s**	Use first worker.
3 / Branch B	**Parse B: time 2-6 s**	Use second worker concurrently.
4 / Join	**Wait until time 7 s**	Compare requires both completed parses.
5 / Finish	**Compare: time 7-9 s**	Final dependent task.
6 / Contrast	**Sequential: 2+5+4+2 = 13 s**	Concurrent elapsed time is 9 s here.

TRANSFER IT TO AN EXAM ANSWER

Speed-up in this idealised case is 13/9 approximately 1.44x, not 2x: the initial load, final comparison and unequal branch times limit gains.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.5(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish concurrency, parallelism and a dependency.

Q2 / 5 MARKS

Draw or describe a dependency plan for Load -> (Parse A, Parse B) -> Compare. Calculate elapsed time with two workers.

Q3 / 6 MARKS

Explain why adding more workers does not make this workflow four times faster. Refer to dependent work and communication overhead.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.5(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Concurrency means tasks make overlapping progress [1]; parallelism means simultaneous execution [1]; dependency means a task needs an earlier result [1].

Q2 / 5

Load is first [1]; A and B can start after Load [1]; comparison waits for both [1]; parallel branch takes max(5,4)=5 s [1]; total 2+5+2=9 s [1].

Q3 / 6

The 2 s load is serial [1]; both parses cannot start earlier [1]; only two parse tasks offer parallel work [1]; compare must await both [1]; synchronisation/communication adds potential overhead [1]; therefore extra workers have diminishing or no benefit for this graph [1].

COMMON MISCONCEPTION

Concurrency is not a promise that every line runs simultaneously; dependent tasks still require ordering.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/

2.1.5(b) / SEE IT / CONCEPT AND VISUAL

A faster booking service can still be wrong

Concurrency can improve responsiveness, but shared mutable data creates risks such as race conditions.

BENEFIT

Handle independent requests while another waits.

RACE

The result depends on unpredictable timing.

PROTECT

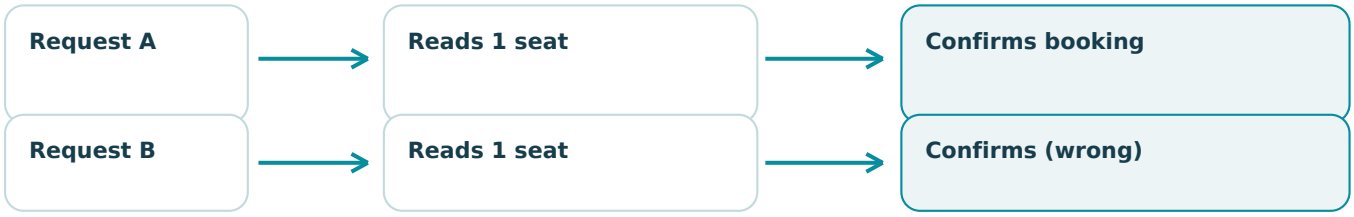
Use atomic operations, locks or transactions.

TRADE-OFF

Synchronisation adds overhead and may reduce throughput.

TWO REQUESTS READING THE SAME LAST SEAT

SHARED STATE: 1 SEAT LEFT



FROM THE DIAGRAM TO EXAM LANGUAGE

If A and B both read availability=1 before either updates it, naïve code might confirm both. The capacity check and decrement must be one protected operation.

WORDS THAT EARN MARKS

- **Race condition** Timing bug
- **Atomic** Indivisible step
- **Lock** Exclusive access
- **Throughput** Work per time

EXAM TRAP

A precondition check is not enough if shared state can change between checking and updating.

2.1.5(b) / WORKED EXAMPLE / STEP BY STEP

Find and repair an overselling race

One seat remains. Requests A and B execute check-then-decrement without a lock.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Shared state	**available = 1**	Both requests share the same count.
2 / A reads	**A observes available = 1**	A decides one place can be sold.
3 / B reads	**B also observes available = 1**	B reaches same decision before A updates.
4 / A commits	**A confirms; available becomes 0**	The last seat is sold.
5 / B commits	**B also confirms; logical result oversold**	Checking earlier was insufficient.
6 / Repair	**Atomic check + decrement**	Second request rechecks protected state and fails.

TRANSFER IT TO AN EXAM ANSWER

A database transaction or suitable lock can keep the test and update together. The serialised critical section can reduce contention-related throughput.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.1.5(b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Identify a race condition and state one benefit and one drawback of concurrent request handling.

Q2 / 5 MARKS

Explain, step by step, how two simultaneous requests can both book the last seat and describe one prevention method.

Q3 / 6 MARKS

Evaluate concurrency for an online booking system, balancing responsiveness, consistency, locking and testing.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.1.5(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Race condition: result depends on interleaving of shared-state operations [1]; benefit: responsiveness/throughput [1]; drawback: race complexity or synchronisation overhead [1].

Q2 / 5

Seat count initially 1 [1]; A reads 1 [1]; B reads 1 before update [1]; both confirm, causing oversell [1]; use atomic check/update or lock/transaction [1].

Q3 / 6

Concurrent requests improve responsiveness [1] and use server resources [1], but shared availability may be inconsistent [1]. Protect the critical section [1]. Locking/transactions incur contention or overhead [1]. Test overlapping requests and check invariant confirmed <= capacity [1].

COMMON MISCONCEPTION

A precondition check is not enough if shared state can change between checking and updating.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval practice: rhsccs.org/revision/h446/section-2-1/